

Using Faker Library



You said:

using faker

ChatGPT said:

It looks like you're asking about **using Faker** (a library to generate fake/mock data). Since you've worked with Node.js + MySQL before, I'll explain it in a **simple + practical way** with examples.

What is Faker?

Faker is a library used to generate **dummy data** like:

- Names
- Emails
- Phone numbers
- Addresses
- Company names

 Useful for:

- Testing apps
- Seeding databases

- Practice projects

Install Faker

For Node.js (modern package)

```
Bash
npm install @faker-js/faker
```

Basic Usage (JavaScript)

```
JavaScript
import { faker } from '@faker-js/faker';

const name = faker.person.fullName();
const email = faker.internet.email();
const city = faker.location.city();

console.log(name);
console.log(email);
console.log(city);
```

Common Faker Methods

User Data

```
JavaScript
faker.person.firstName();
faker.person.lastName();
faker.person.fullName();
```

Internet

```
JavaScript
faker.internet.email();
faker.internet.username();
faker.internet.password();
```

Address

```
JavaScript
faker.location.city();
faker.location.streetAddress();
faker.location.country();
```

Phone

```
JavaScript
faker.phone.number();
```

Generate Multiple Fake Users

```
JavaScript
import { faker } from '@faker-js/faker';

function createUser() {
  return {
    name: faker.person.fullName(),
    email: faker.internet.email(),
    age: faker.number.int({ min: 18, max: 60 })
  };
}

const users = [];

for (let i = 0; i < 5; i++) {
  users.push(createUser());
}

console.log(users);
```



Insert Fake Data into MySQL (Your Use Case)

```
JavaScript
import mysql from 'mysql2';
import { faker } from '@faker-js/faker';

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'testdb'
});

for (let i = 0; i < 10; i++) {
  const name = faker.person.fullName();
  const email = faker.internet.email();

  const query = 'INSERT INTO users (name, email) VALUES (?, ?)';

  connection.query(query, [name, email], (err) => {
    if (err) throw err;
  });
}

console.log("Fake data inserted!");
```



Python Version (if needed)

```
Bash
pip install faker
```

```
Python
from faker import Faker

fake = Faker()

print(fake.name())
print(fake.email())
print(fake.city())
```



Pro Tips (Important for Interviews / Projects)

- Use Faker for **database seeding**
- Combine with loops → generate thousands of records

- Great for **testing APIs**
- Helps avoid using real user data (privacy safe)

⚠️ Common Mistake

If you get error like:

```
Cannot use import statement outside module
```

👉 Fix:

- Add in **package.json**

```
JSON
"type": "module"
```

OR use:

```
JavaScript
const { faker } = require('@faker-js/faker');
```

If you want, I can help you:

- ✓ Build a **full database seeder script**
- ✓ Generate **1 lakh+ records efficiently**
- ✓ Integrate Faker into your **Express project**

Just tell me 👍

You said:

mysql package

ChatGPT said:

Got it — you want to understand the **MySQL package in Node.js** (very important for your Express + MySQL project).

I'll explain it **clearly + practically** 📌



What is MySQL package in Node.js?

The MySQL package lets your Node.js app **connect and interact with a MySQL database**.

👉 You can:

- Insert data
- Read data
- Update data
- Delete data



Which package should you use?

✅ Best (Recommended)

```
Bash  
npm install mysql2
```

👉 Why **mysql2**?

- Faster
- Supports **Promises (modern async/await)**

- Better than old `mysql` package

Basic Connection Example

```
JavaScript
import mysql from 'mysql2';

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'your_password',
  database: 'testdb'
});

connection.connect((err) => {
  if (err) {
    console.log('Error connecting:', err);
    return;
  }
  console.log('Connected to MySQL');
});
```

Insert Data

```
JavaScript
const query = 'INSERT INTO users (name, email) VALUES (?, ?)';

connection.query(query, ['Aamir', 'aamir@gmail.com'], (err,
result) => {
  if (err) throw err;
  console.log('Data inserted');
});
```

Fetch Data

```
JavaScript
connection.query('SELECT * FROM users', (err, results) => {
  if (err) throw err;
  console.log(results);
});
```



Update Data

```
JavaScript
connection.query(
  'UPDATE users SET name = ? WHERE id = ?',
  ['New Name', 1],
  (err) => {
    if (err) throw err;
    console.log('Updated');
  }
);
```



Delete Data

```
JavaScript
connection.query(
  'DELETE FROM users WHERE id = ?',
  [1],
  (err) => {
    if (err) throw err;
    console.log('Deleted');
  }
);
```


Best Practice (IMPORTANT)

Use Promise Version (Clean Code)

```
JavaScript
import mysql from 'mysql2/promise';

const db = await mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'your_password',
  database: 'testdb'
});

// Insert
await db.execute(
  'INSERT INTO users (name, email) VALUES (?, ?)',
  ['Aamir', 'test@gmail.com']
);

// Fetch
const [rows] = await db.execute('SELECT * FROM users');
console.log(rows);
```

Combine with Faker (Power Combo)

```
JavaScript
import mysql from 'mysql2/promise';
import { faker } from '@faker-js/faker';

const db = await mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'your_password',
  database: 'testdb'
});

for (let i = 0; i < 10; i++) {
  await db.execute(
    'INSERT INTO users (name, email) VALUES (?, ?)',
    [
      faker.person.fullName(),
      faker.internet.email()
    ]
  );
}

console.log("Fake users inserted!");
```

Common Errors + Fix

ER_ACCESS_DENIED_ERROR

 Wrong username/password

Unknown database

 Database not created

```
SQL
CREATE DATABASE testdb;
```

Table doesn't exist

 Create table first

```
SQL
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);
```

Interview Important Points

- Difference between `mysql` vs `mysql2`
- What are prepared statements (?)
- Why use `async/await`
- How to prevent SQL injection (use placeholders)

If you want next step, I can help you:

- ✓ Build **complete backend (Express + MySQL + Faker)**
- ✓ Create **real project like Zerodha clone DB**
- ✓ Give **50 SQL + Node interview questions with answers**

Just tell me 👍

You said:

using sql from CLI

ChatGPT said:

Got it — you want to **use MySQL from the CLI (Command Line Interface)**. This is super important for interviews and real development. I'll walk you step-by-step in a clean, practical way 🙌

1. Open MySQL CLI

👉 On Windows (CMD / PowerShell)

```
Bash
mysql -u root -p
```

👉 Then enter your password

If command not found

Use full path:

```
Bash
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u
root -p
```

2. Basic Commands (VERY IMPORTANT)

◆ Show Databases

```
SQL
SHOW DATABASES;
```

◆ Create Database

```
SQL
CREATE DATABASE testdb;
```

◆ Use Database

```
SQL
USE testdb;
```

◆ Show Tables

```
SQL
SHOW TABLES;
```

3. Create Table

```
SQL
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);
```



4. Insert Data

```
SQL
INSERT INTO users (name, email)
VALUES ('Aamir', 'aamir@gmail.com');
```



5. Fetch Data

```
SQL
SELECT * FROM users;
```



6. Update Data

```
SQL
UPDATE users
SET name = 'Updated Name'
WHERE id = 1;
```



7. Delete Data

```
SQL
DELETE FROM users
WHERE id = 1;
```



8. Useful CLI Shortcuts

◆ Clear screen

```
SQL
\! cls
```

◆ Exit MySQL

```
SQL
exit;
```

9. Import SQL File (VERY USEFUL)

```
Bash
mysql -u root -p testdb < file.sql
```

10. Export Database

```
Bash
mysqldump -u root -p testdb > backup.sql
```

11. Generate Bulk Data (Manual Way)

```
SQL
INSERT INTO users (name, email)
VALUES
('User1', 'u1@gmail.com'),
('User2', 'u2@gmail.com'),
('User3', 'u3@gmail.com');
```

Common Errors

Access denied

 Wrong password

Unknown database

 Run:

```
SQL
CREATE DATABASE testdb;
```

Table doesn't exist

 Check:

```
SQL
SHOW TABLES;
```

Pro Tips (Interview + Real Use)

- Always end SQL with ;
- Use `DESC users;` to see table structure
- Use `WHERE` carefully (avoid deleting all data 😬)
- Practice directly in CLI → improves speed

Next Level (If you want)

I can help you:

- ✓ Practice **50 SQL CLI questions (interview level)**
- ✓ Learn **joins, subqueries, advanced SQL**
- ✓ Connect CLI SQL with your **Node.js project**

Just tell me 👍

You said:

CREATE TABLE USER

ChatGPT said:

Close — but **USER** is a reserved keyword in MySQL, so using it directly can cause errors.

👉 Don't do this:

```
SQL
CREATE TABLE USER (...);
```

✓ Correct Ways

✓ Option 1 (Best Practice: rename table)

```
SQL
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);
```

👉 ✓ Recommended → use **users** (plural)

✓ Option 2 (Use backticks if you really want **USER**)

```
SQL
CREATE TABLE `USER` (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);
```

👉 ⚠ Not recommended (can confuse later)

💡 Why this error happens?

- **USER** is used internally in MySQL (like system keyword)
- MySQL gets confused between:
 - keyword **USER**
 - your table name

🔍 Verify Table

```
SQL
SHOW TABLES;
```


Check Structure

```
SQL
DESC users;
```

Pro Tip (Important for Interviews)

👉 Always avoid reserved words:

- USER
- ORDER
- GROUP
- SELECT

👉 Use:

- `users`
- `orders`
- `groups`

If you want, I can give you:

- ✓ Full **MySQL schema design (like real projects)**
- ✓ **50 table creation practice questions**
- ✓ Advanced constraints (FK, UNIQUE, etc.)

Just tell me 👉

You said:

Give me 50 table creation practice questions

ChatGPT said:

Perfect — this is exactly how you become strong in SQL 🙌

I'll give you **50 table creation practice questions** from **easy** → **advanced** → **real interview level**.

👉 Your task: **Write CREATE TABLE queries for each**

EASY LEVEL (1–15)

1

Create a table **students** with:

- id (PK, auto increment)
- name
- age

2

Create **employees**:

- id (PK)
- name
- salary (decimal)

3

Create **products**:

- id (PK)
- product_name
- price

4

Create **customers**:

- id (PK)
- name
- email (unique)

5

Create **books**:

- id (PK)
- title

- author
- price

6

Create **orders**:

- id (PK)
- order_date
- total_amount

7

Create **cities**:

- id (PK)
- city_name
- state

8

Create **courses**:

- id (PK)
- course_name
- duration

9

Create **teachers**:

- id (PK)
- name
- subject

10

Create **departments**:

- id (PK)

- dept_name

11

Create **vehicles**:

- id (PK)
- brand
- model

12

Create **movies**:

- id (PK)
- title
- release_year

13

Create **songs**:

- id (PK)
- title
- artist

14

Create **countries**:

- id (PK)
- country_name
- population

15

Create **accounts**:

- id (PK)
- username (unique)

- password

MEDIUM LEVEL (16–35)

16

Create **students** with:

- id (PK)
- name (NOT NULL)
- email (UNIQUE)
- age (CHECK > 18)

17

Create **employees**:

- id (PK)
- name
- salary (DEFAULT 10000)

18

Create **products**:

- id (PK)
- name
- price (CHECK > 0)

19

Create **orders**:

- id (PK)
- customer_id (FK)

- order_date

20

Create **order_items**:

- id (PK)
- order_id (FK)
- product_id (FK)
- quantity

21

Create **users**:

- id (PK)
- username (UNIQUE, NOT NULL)
- password (NOT NULL)
- created_at (DEFAULT current timestamp)

22

Create **payments**:

- id (PK)
- amount
- payment_date
- status (ENUM: success, failed, pending)

23

Create **attendance**:

- id (PK)
- student_id (FK)
- date

- status (present/absent)

24

Create **reviews**:

- id (PK)
- product_id (FK)
- rating (1–5)
- comment

25

Create **categories**:

- id (PK)
- name (UNIQUE)

26

Create **inventory**:

- id (PK)
- product_id (FK)
- stock_quantity

27

Create **addresses**:

- id (PK)
- user_id (FK)
- city
- pincode

28

Create **blogs**:

- id (PK)

- title
- content
- created_at

29

Create **comments**:

- id (PK)
- blog_id (FK)
- user_id (FK)
- comment_text

30

Create **likes**:

- id (PK)
- user_id (FK)
- post_id (FK)

31

Create **roles**:

- id (PK)
- role_name

32

Create **user_roles**:

- id (PK)
- user_id (FK)

- role_id (FK)

33

Create **sessions**:

- id (PK)
- user_id (FK)
- login_time
- logout_time

34

Create **notifications**:

- id (PK)
- user_id (FK)
- message
- is_read (boolean)

35

Create **files**:

- id (PK)
- file_name
- file_size
- uploaded_at

HARD LEVEL (36–50)

36

Create **students** + **courses** + **enrollments** (many-to-many)

37

Create **employees** with:

- manager_id (self FK)

38

Create **transactions**:

- id
- sender_id
- receiver_id
- amount

39

Create **subscriptions**:

- user_id
- plan
- start_date
- end_date

40

Create **shopping_cart**:

- user_id
- product_id
- quantity

41

Create **wishlist**:

- user_id

- product_id

42

Create **messages**:

- sender_id
- receiver_id
- message
- timestamp

43

Create **audit_logs**:

- id
- action
- user_id
- timestamp

44

Create **payments**:

- id
- user_id
- amount
- method (UPI, card, cash)

45

Create **delivery**:

- order_id
- status

- delivery_date

46

Create **tickets**:

- id
- user_id
- issue
- status

47

Create **support_responses**:

- ticket_id
- response
- responder_id

48

Create **events**:

- id
- name
- location
- date

49

Create **event_participants**:

- event_id
- user_id

50

Create full **e-commerce schema**:

- users

- products
- orders
- order_items
- payments

How to Practice (Important)

👉 Do like this:

1. Try yourself
2. Write query in CLI
3. Run it
4. Fix errors

Next Step

If you want, I can:

- ✅ Give **all 50 solutions step-by-step**
- ✅ Take you **question by question (interview style)**
- ✅ Give **real-world DB design (Zerodha / E-commerce)**

Just tell me 👍

You said:

Give all 50 solutions step-by-step with ques
2/2

ChatGPT said:

Great — now we'll do this **properly step-by-step with questions + solutions** (like interview prep).

I'll keep queries **clean, correct, and production-style (MySQL)**.

EASY (1–15)

1. Create **students** (id, name, age)

```
SQL
CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  age INT
);
```

2. Create **employees** (id, name, salary)

```
SQL
CREATE TABLE employees (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  salary DECIMAL(10,2)
);
```

3. Create **products**

```
SQL
CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(100),
  price DECIMAL(10,2)
);
```

4. Create **customers** (email unique)

```
SQL
CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100) UNIQUE
);
```

5. Create **books**

```
SQL
CREATE TABLE books (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(150),
  author VARCHAR(100),
  price DECIMAL(10,2)
);
```

6. Create **orders**

```
SQL
CREATE TABLE orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_date DATE,
  total_amount DECIMAL(10,2)
);
```

7. Create **cities**

```
SQL
CREATE TABLE cities (
  id INT AUTO_INCREMENT PRIMARY KEY,
  city_name VARCHAR(100),
  state VARCHAR(100)
);
```

8. Create **courses**

```
SQL
CREATE TABLE courses (
  id INT AUTO_INCREMENT PRIMARY KEY,
  course_name VARCHAR(100),
  duration INT
);
```

9. Create **teachers**

```
SQL
CREATE TABLE teachers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  subject VARCHAR(100)
);
```

10. Create **departments**

```
SQL
CREATE TABLE departments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  dept_name VARCHAR(100)
);
```

11. Create **vehicles**

```
SQL
CREATE TABLE vehicles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  brand VARCHAR(100),
  model VARCHAR(100)
);
```

12. Create **movies**

```
SQL
CREATE TABLE movies (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(150),
  release_year YEAR
);
```


13. Create **songs**

```
SQL
CREATE TABLE songs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(150),
  artist VARCHAR(100)
);
```

14. Create **countries**

```
SQL
CREATE TABLE countries (
  id INT AUTO_INCREMENT PRIMARY KEY,
  country_name VARCHAR(100),
  population BIGINT
);
```

15. Create **accounts**

```
SQL
CREATE TABLE accounts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(100) UNIQUE,
  password VARCHAR(255)
);
```

MEDIUM (16–35)

16. Students with constraints

```
SQL
CREATE TABLE students2 (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE,
  age INT CHECK (age > 18)
);
```

17. Employees with default salary

```
SQL
CREATE TABLE employees2 (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  salary DECIMAL(10,2) DEFAULT 10000
);
```

18. Products with price check

```
SQL
CREATE TABLE products2 (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  price DECIMAL(10,2) CHECK (price > 0)
);
```

19. Orders with FK

```
SQL
CREATE TABLE orders2 (
  id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT,
  order_date DATE,
  FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

20. Order items

```
SQL
CREATE TABLE order_items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT,
  product_id INT,
  quantity INT,
  FOREIGN KEY (order_id) REFERENCES orders2(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```

21. Users with timestamp

```
SQL
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(100) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

22. Payments with ENUM

```
SQL
CREATE TABLE payments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  amount DECIMAL(10,2),
  payment_date DATE,
  status ENUM('success','failed','pending')
);
```

23. Attendance

```
SQL
CREATE TABLE attendance (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT,
  date DATE,
  status ENUM('present','absent'),
  FOREIGN KEY (student_id) REFERENCES students(id)
);
```

24. Reviews

```
SQL
CREATE TABLE reviews (
  id INT AUTO_INCREMENT PRIMARY KEY,
  product_id INT,
  rating INT CHECK (rating BETWEEN 1 AND 5),
  comment TEXT,
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```

25. Categories

```
SQL
CREATE TABLE categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) UNIQUE
);
```

26. Inventory

```
SQL
CREATE TABLE inventory (
  id INT AUTO_INCREMENT PRIMARY KEY,
  product_id INT,
  stock_quantity INT,
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```

27. Addresses

```
SQL
CREATE TABLE addresses (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  city VARCHAR(100),
  pincode VARCHAR(10),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

28. Blogs

```
SQL
CREATE TABLE blogs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(200),
  content TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

29. Comments

```
SQL
CREATE TABLE comments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  blog_id INT,
  user_id INT,
  comment_text TEXT,
  FOREIGN KEY (blog_id) REFERENCES blogs(id),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

30. Likes

```
SQL
CREATE TABLE likes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  post_id INT,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

31. Roles

```
SQL
CREATE TABLE roles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  role_name VARCHAR(100)
);
```

32. User roles

```
SQL
CREATE TABLE user_roles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  role_id INT,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (role_id) REFERENCES roles(id)
);
```

33. Sessions

```
SQL
CREATE TABLE sessions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  login_time DATETIME,
  logout_time DATETIME,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

34. Notifications

```
SQL
CREATE TABLE notifications (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  message TEXT,
  is_read BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

35. Files

```
SQL
CREATE TABLE files (
  id INT AUTO_INCREMENT PRIMARY KEY,
  file_name VARCHAR(255),
  file_size INT,
  uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

HARD (36–50)

36. Many-to-many (students, courses, enrollments)

```
SQL
CREATE TABLE enrollments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT,
  course_id INT,
  FOREIGN KEY (student_id) REFERENCES students(id),
  FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

37. Employees with manager (self FK)

```
SQL
CREATE TABLE employees3 (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  manager_id INT,
  FOREIGN KEY (manager_id) REFERENCES employees3(id)
);
```

38. Transactions

```
SQL
CREATE TABLE transactions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  sender_id INT,
  receiver_id INT,
  amount DECIMAL(10,2)
);
```

39. Subscriptions

```
SQL
CREATE TABLE subscriptions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  plan VARCHAR(50),
  start_date DATE,
  end_date DATE,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

40. Shopping cart

```
SQL
CREATE TABLE shopping_cart (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  product_id INT,
  quantity INT,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```

41. Wishlist

```
SQL
CREATE TABLE wishlist (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  product_id INT,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```


42. Messages

```
SQL
CREATE TABLE messages (
  id INT AUTO_INCREMENT PRIMARY KEY,
  sender_id INT,
  receiver_id INT,
  message TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

43. Audit logs

```
SQL
CREATE TABLE audit_logs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  action VARCHAR(255),
  user_id INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

44. Payments (method)

```
SQL
CREATE TABLE payments2 (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  amount DECIMAL(10,2),
  method ENUM('UPI','card','cash'),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

45. Delivery

```
SQL
CREATE TABLE delivery (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT,
  status VARCHAR(50),
  delivery_date DATE,
  FOREIGN KEY (order_id) REFERENCES orders2(id)
);
```

46. Tickets

```
SQL
CREATE TABLE tickets (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  issue TEXT,
  status VARCHAR(50),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

47. Support responses

```
SQL
CREATE TABLE support_responses (
  id INT AUTO_INCREMENT PRIMARY KEY,
  ticket_id INT,
  response TEXT,
  responder_id INT,
  FOREIGN KEY (ticket_id) REFERENCES tickets(id)
);
```

48. Events

```
SQL
CREATE TABLE events (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150),
  location VARCHAR(150),
  event_date DATE
);
```

49. Event participants

```
SQL
CREATE TABLE event_participants (
  id INT AUTO_INCREMENT PRIMARY KEY,
  event_id INT,
  user_id INT,
  FOREIGN KEY (event_id) REFERENCES events(id),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

50. E-commerce schema (core tables)

```
SQL
CREATE TABLE ecommerce_users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100) UNIQUE
);

CREATE TABLE ecommerce_products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  price DECIMAL(10,2)
);

CREATE TABLE ecommerce_orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  order_date DATE,
  FOREIGN KEY (user_id) REFERENCES ecommerce_users(id)
);

CREATE TABLE ecommerce_order_items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT,
  product_id INT,
  quantity INT,
  FOREIGN KEY (order_id) REFERENCES ecommerce_orders(id),
  FOREIGN KEY (product_id) REFERENCES ecommerce_products(id)
);

CREATE TABLE ecommerce_payments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT,
  amount DECIMAL(10,2),
  FOREIGN KEY (order_id) REFERENCES ecommerce_orders(id)
);
```



What Next?

Now you should:

1. Run each query in CLI
2. Fix errors (this is where real learning happens)

3. Try modifying tables

If you want next level:

- ✓ 50 **ALTER TABLE** questions
- ✓ 50 **INSERT + SELECT** queries practice
- ✓ Full **real-world DB design (Zerodha / Instagram clone)**

Just tell me 👍

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).